

Exemple d'affichage complet

Objectifs

- Synthétiser les éléments clés d'un chapitre.
- Illustrer avec un extrait de code concis.
- Donner 1 à 2 repères visuels (tableau, schéma).

Code Python (basique)

```
def somme(liste):  
    total = 0  
    for x in liste:  
        total += x  
    return total  
  
<p>print(somme([1, 2, 3])) # 6
```

Définitions

Algorithme : suite finie d'instructions permettant de résoudre un problème.
Complexité : quantité de ressources (temps, mémoire) consommées par un algorithme.

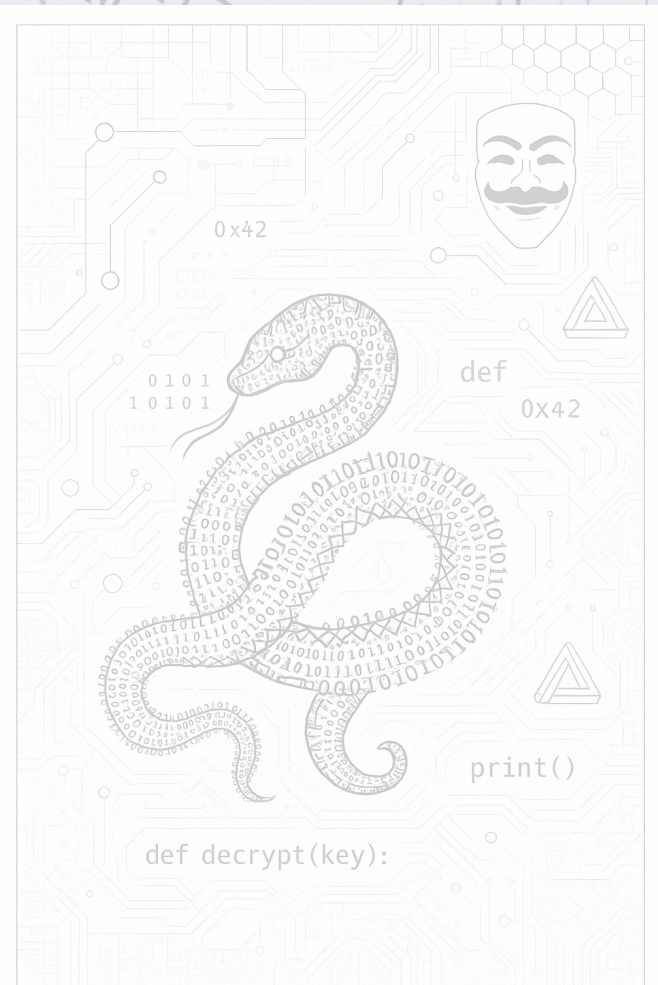
Idiomes Python

```
def evenes():  
    evens = [x for x in range(10) if x % 2 == 0]  
    return evens  
  
<p>print(evenes()) # [0, 2, 4, 6, 8]  
  
<p>print(s[::-1]) # ISN
```

Tableau compact

Structure	Mutable	Clés/Index
list	Oui	Index entiers
tuple	Non	Index entiers
dict	Oui	Clés hashables

Schéma (image)



Note

Astuce : préférez plusieurs petites sections à une seule très dense.
Voir aussi : [Documentation Python](#).

Exemple de mise en page A4 optimisée

HACK
AGAINST
MACHISM

Les chaînes de caractères en Python

Définitions & Propriétés

- Une chaîne est une suite de caractères entre guillemets.
- Type immuable : on ne peut pas modifier une chaîne après sa création.

```
root@python:~$  
python3  
Python 3.6.3  
mot[0] = "J" # Erreur !
```

+ Opérations de base

```
root@python:~$  
python3  
Python 3.6.3  
print(s[1])
```

Fiche NSI - Chaînes de caractères - Terminale

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM

Les Listes en Python

💡 Définition et Création

Une liste est une collection de données ordonnée et modifiable. C'est l'une des structures de données les plus importantes en Python. Elle est reconnaissable par les crochets `[]` qui l'entourent. On peut y stocker des éléments de types différents (nombres, chaînes de caractères, booléens, etc.).

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste vide
>>> fruits = []
>>> # Ajoute des éléments de types différents
>>> fruits.append("pomme")
>>> fruits.append(2)
>>> fruits.append(True)
>>> # Vérifie la longueur d'une liste avec len()
>>> len(fruits)
3
>>> print(f"La liste 'fruits' contient {len(fruits)} éléments.") # La liste 'fruits' contient 3 éléments.
```

La compréhension de liste (List Comprehension)

C'est une syntaxe rapide et très "pythonique" pour créer des listes à partir d'autres collections. Elle combine une boucle et une condition sur une seule ligne. C'est une astuce de Python que vous rencontrerez souvent.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste de carrés pairs de 0 à 10
>>> carres_pairs = [x**2 for x in range(0, 11) if x % 2 == 0]
>>> print(carres_pairs) # Affiche [0, 4, 16, 36, 64, 100]
```

⚙️ Modification de la liste

Les listes sont mutables, ce qui signifie qu'on peut ajouter ou supprimer des éléments après leur création.

Ajouter des éléments

- `append(element)`: ajoute un élément à la fin de la liste.
- `insert(index, element)`: insère un élément à un index précis.
- `extend(iterable)`: ajoute tous les éléments d'un autre objet (comme une autre liste) à la fin.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste de jours de la semaine
>>> jours = ["lundi", "mardi", "jeudi", "vendredi", "samedi", "dimanche"]
>>> # Ajoute un jour à la fin
>>> jours.append("dimanche")
>>> # Insère un jour à un index précis
>>> jours.insert(2, "jeudi")
>>> # Ajoute une autre liste à la fin
>>> jours.extend(["mercredi", "jeudi", "vendredi"])
>>> print(jours) # ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche']
```

Supprimer des éléments

- `del ma_liste[index]`: supprime l'élément à l'index donné.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste de nombres
>>> nombres = [10, 20, 30, 40]
>>> # Supprime l'élément à index 1 (le nombre 20)
>>> del nombres[1]
>>> print(nombres) # [10, 30, 40]
```

- `pop(index)`: supprime l'élément à l'index donné et le renvoie. Si l'index est omis, il supprime et renvoie le dernier élément.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste de nombres
>>> nombres = [10, 20, 30, 40]
>>> # Supprime l'élément à l'index 1 et le renvoie
>>> element_supprime = nombres.pop(1)
>>> print(nombres) # [10, 30]
```

📋 Copier une liste

Si vous attribuez une liste à une autre variable avec le signe `=`, vous ne faites pas une copie ! Les deux variables pointent vers le même objet en mémoire. On appelle cela une référence.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste
>>> liste_a = [1, 2, 3, 4]
>>> # Crée une autre variable qui pointe vers la même liste
>>> liste_b = liste_a
>>> # C'est une référence, pas une copie !
>>> liste_b[0] = 5
>>> print(liste_a) # Affiche [5, 2, 3, 4] !
```

Pour faire une vraie copie, on peut utiliser la technique du slicing ou la méthode `.copy()`.

🔍 Indexation et Accès aux éléments

Chaque élément d'une liste possède une position, appelée index, qui commence à 0.

On peut aussi utiliser des index négatifs pour accéder aux éléments de la liste. L'index -1 correspond au dernier élément, -2 à l'avant-dernier, etc.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste d'animaux
>>> animaux = ["chat", "oiseau", "poisson"]
>>> # Accède à l'élément à l'index 1
>>> animaux[1]
'oiseau'
>>> # Accède à l'élément à l'index 2
>>> animaux[2]
'poisson'
>>> # Accède à l'élément à l'index négatif -1 (le dernier élément)
>>> animaux[-1]
'poisson'
>>> # Accède à l'élément à l'index négatif -2 (l'avant-dernier élément)
>>> animaux[-2]
'oiseau'
>>> print(animaux[2:]) # Affiche ['oiseau', 'poisson']
```

🔄 Parcourir une liste

Une boucle `for` est l'outil principal pour parcourir les éléments d'une liste.

1. Par élément :

C'est la méthode la plus simple et la plus courante. On parcourt chaque élément de la liste.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste de pizzas
>>> pizzas = ["calzone", "pepperoni"]
>>> # Parcourt chaque pizza
>>> for pizza in pizzas:
>>>     print(pizza)
calzone
pepperoni
```

2. Par index :

Utile si vous avez besoin de la position de l'élément en plus de l'élément lui-même. On utilise la fonction `range(len(liste))`.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste de pizzas
>>> pizzas = ["calzone", "pepperoni"]
>>> # Parcourt chaque pizza avec son index
>>> for i in range(len(pizzas)):
>>>     print(f"Pizza n°{i} : {pizzas[i]}")
Pizza n°0 : calzone
Pizza n°1 : pepperoni
```

3. Parcours mixte (avec `enumerate`) :

La méthode la plus élégante pour avoir à la fois l'index et l'élément. Elle est plus claire que la méthode précédente.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Crée une liste de pizzas
>>> pizzas = ["calzone", "pepperoni"]
>>> # Parcourt chaque pizza avec son index
>>> for index, pizza in enumerate(pizzas):
>>>     print(f"Index {index} correspond à la {pizza}.")
Index 0 correspond à la calzone.
Index 1 correspond à la pepperoni.
```



```
montanthon>$
11:00:00 [1:] # Vraie copie avec slicing
11:00:00
11:00:00 [1:] # Affiche [1, 2, 3, 4]
print(liste_c) # Affiche [1, 2, 3, 4]
```

Fiche d'aide-mémoire - NSI Tani Mallandi

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM

Les Structures de Données

Les Listes (list)

- Collection ordonnée et modifiable d'éléments.
- Permet les doublons.

```
ma_liste = ["a", 3.14, 1]
print(ma_liste[1]) # a
```

Les Dictionnaires (dict)

Collection non-ordonnée de paires clé-valeur. Mutable.

```
personne = {"nom": "Alan", "age": 41}
print(personne["nom"]) # Alan
```

Fiche NSI - Structures de Données - Terminale

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM